Universidad del Valle de Guatemala
CC-2008 – Programación Orientada a Objetos
Semestre II, 2026



DOCUMENTACION

Integrantes:
Luis Ángel Valiente Osorio, 261288
Andreh Alexander Callejas Ajin 261828
Manuel Alejandro Rio Oliva 16639

Recuento de Requisitos Funcionales

Luis Ángel Valiente Osorio

Entrega	Avance Realizado	Resultado acumulado
Fase 3	Desarrollo de RF07 – Tareas: Representación. registro y consulta de tareas.	Modulo inicial de gestión de Tareas
Fase 4	Comienzo y finalización de la base de datos.	Modelado inicial de la base de datos y sus conexiones
Fase 5	Retirar persistencia y Terminar Conexiones a la base de datos. Separación de sesiones y verificación con conexiones	Tareas persistentes y datos vinculados en MySQL. Registro, consulta y finalización. Integración comprobada con los modulos

Planificación

RF07 – Tareas y Controlador / Base de datos. Responsable: Luis Ángel Valiente Osorio

Tarea	Descripción	Prioridad
Desarrollar RF07	Representar las tareas y permitir su registro, la consulta y la finalización	Alta
Integrar MySQL	Conectar java con MySQL mediante un JDBC y relacionar las tareas, usuarios y cursos	Alta
Unificar la persistencia e incorporar Prioridad	Se había retirado la prioridad, entonces se volvió a incorporar. Dejar lista la base de datos con sus respectivas tablas	Media
Retirar persistencia antigua	Retirar la persistencia antigua que se guardaba mediante CSV y dejar listo solo el MySQL.	Media
Verificar integración con MySQL	Realizar pruebas desde el formulario para ver si pasan usuarios, tareas, cursos y calificaciones a la base de datos.	Baja

Implementación del sistema

Luis Ángel Valiente Osorio

Trabajo principal en el registro de tareas al igual que sus consultas. Responsable de la base de datos. Principales componentes en los cuales se trabajó:

Tarea: Representa los datos y el estado de una tarea

PersistenciaTareasMySQl: inserta, consulta y actualiza tareas en MySQL

ServidorWeb: Se alteraron las sesiones. Ahora son separadas, sustituyendo un usuario global por sesiones identificadas mediante tokens y cookies. Ahora las sesiones son temporales y los datos Ingresados se guardan en MySQL

Integración con la base de datos y su verificación: Configure el acceso a MySQL mediante un JDBC, se trabajo un Docker para por ejecutar la base de datos y para tratar con las consultas de una manera más cómoda se utilizó DBeaver.

Se corrigieron algunos bugs:

- Consulta duplicada que impedía cargar las tareas

- SELECT que estaba ubicado incorrectamente dentro de la función guardarTarea.

Ahora se utiliza una sola forma de persistencia; retirando el almacenamiento con CSV y en memoria, dejando solamente persistencia con MySQL

Verifaciones:

-Se empezó compilando la aplicación con las pruebas.

- Revisión del registro de tareas y calificaciones desde la interfaz y consulta con los usuarios en DBeaver.

- Verificación de las nuevas funciones para usuario y cursos implementadas.

Reflexión

Luis Angel Valiente Osorio

Una de las principales dificultades con la que me tope fue la integración de mi modulo (RF07) con el MySQL. Existían varias formas de persistencia, lo cual me dificultaba saber dónde íbamos a guardar la información. En algunas consultas del SQL, una impedía cargar y guardar las tareas, entonces se cambio el SELECT por un INSERT. Entonces el solo compilar con java no basta, aprendí a tener una fuente consistente de información y verificar siempre al compilar que si funcione lo que se compile.

Una dificultad fue el servidor, ya que usaba un único usuario para diferentes navegadores. En este vaso se requirió coordinación de mis compañeros para adaptar las nuevas sesiones mediante tokens y así tener sesiones temporales en cada una. Me llevo para una próxima identificar los usuarios en cada solicitud.

Gestión / Calendario de Tiempos

Fecha	Inicio	Fin	Interrupción	Delta T	Tarea	Comentario
27/10/2026	10:00	16:00	340 - 90	270	Borrar persistencia antigua	Trabajé afuera, por ende, almorcé por un rato
31/10/2026	18:00	22:00	240 - 50	190	Conexiones a MySQL y Parches de bugs	Tuve el día libre y me puse a leer un rato
01/11/2026	17:00	17:00	120 - 20	100	Verificación de pruebas en MySQL	Descanso para jugar Minecraft

Recuento de Requisitos Funcionales

Fabricio Xiloj - 262140

Entrega	Avance Realizado	Resultado acumulado
Fase 3	RF03 – Registro de calificaciones, guardas y registrar calificaciones	Apartado de calificaciones con guardado temporal en el programa
Fase 4	RF03 – Mejora de Registro de calificaciones, junto a cursos, se integraron para que fueran funcionales entre si. Mejoras en el repositorio	Apartado de cursos, en complemento de calificaciones.
Fase 5	Integración por completo a SQL, Cursos calificaciones, y se trabajó en parches (rama parches) Se creo la función de usuarios, funcional con índex y SQL	Todo integrado y listo al SQL, parches arreglados, y calificaciones y cursos completados, también se logró crear usuarios, con su interfaz y conexión a SQL

Planificación

RF07 – Tareas y Controlador / Base de datos. Responsable: Luis Ángel Valiente Osorio

Tarea	Descripción	Prioridad
Desarrollar RF03	Crear el apartado para registrar y consultar calificaciones.	Alta
Validar calificaciones	Comprobar que las notas sean válidas y que no superen el punteo máximo.	Alta
Integrar calificaciones con cursos	Relacionar las calificaciones con los cursos registrados y mostrarlas de forma ordenada.	Alta
Implementar usuarios	Preparar el registro de usuarios y conectarlo con el inicio de sesión.	Alta
Corregir parches	Revisar y ajustar las pruebas de login, tareas, cursos y dashboard, junto a pruebas	Media
Preparar integración con SQL	Dejar los archivos Java y las pruebas listas para que funcionen con la base de datos.	Media
Verificar el sistema	Comprobar la compilación y el funcionamiento de calificaciones, cursos, usuarios y pruebas.	Alta

Implementación del sistema

Implementación del sistema

La implementación del sistema se realizó de forma gradual, incorporando nuevas funciones en cada fase del proyecto. Primero se desarrolló el apartado de calificaciones, donde el usuario podía registrar una nota indicando el curso, la actividad, el resultado obtenido, el punteo máximo y la fecha. El sistema validaba que los datos fueran correctos y permitía consultar las calificaciones guardadas.

Después se integró el apartado de cursos con las calificaciones. Permitió relacionar cada nota con un curso específico y consultar la información de manera más organizada. También se mejoró la forma en que se guardaban y mostraban los datos dentro del sistema.

En la última fase se conectó el proyecto con la base de datos SQL. Se incorporaron los cambios necesarios para guardar la información de manera permanente, se agregaron funciones para registrar usuarios y se realizaron diferentes correcciones en la rama de parches. Las calificaciones, los cursos, las tareas y los usuarios quedaron listos para la base de datos.

La implementación se hizo de manera progresiva, verificando cada función mediante pruebas y realizando ajustes para mantener la compatibilidad entre las diferentes partes del sistema y sus archivos. Como resultado, se consiguió una plataforma más organizada, junto a un repositorio más ordenado y con cambios más notorios a la primera entrega de código.

Verificaciones

Se realizaron pruebas para comprobar que las funciones principales respondieran correctamente. Se verificó el registro y consulta de calificaciones, el funcionamiento de tareas, cursos, promedios, login y servidores. También se revisó que el proyecto compilara correctamente y que la interfaz mantuviera una conexión con las funciones agregadas.

Calificaciones

Se verificó que las calificaciones pudieran registrarse con sus datos correspondientes. También se comprobó que no se aceptaran notas negativas, notas mayores al punteo máximo ni campos incompletos.

Parches

Se revisaron y corrigieron las pruebas relacionadas con login, tareas, cursos y dashboard. También se ajustó el proyecto para que las diferentes partes funcionaran correctamente después de integrar los cambios de tareas y base de datos.

Usuarios

Se verificó el registro de nuevos usuarios y su conexión con el inicio de sesión. También se comprobaron los campos obligatorios y se dejaron preparados los archivos necesarios para trabajar con la base de datos.

Reflexión

Durante el desarrollo del proyecto aprendí que una aplicación no se construye únicamente escribiendo código, sino también organizando el trabajo, realizando pruebas y revisando constantemente que las diferentes partes funcionen juntas. La implementación de las calificaciones me permitió comprender mejor cómo se reciben, validan y muestran los datos dentro del sistema.

También aprendí la importancia de trabajar por ramas y mantener una buena comunicación con el equipo. Al integrar cursos, usuarios, tareas y la base de datos surgieron algunos errores, pero estos ayudaron a identificar la necesidad de realizar pruebas y hacer correcciones antes de unir los cambios al proyecto principal. (o al main)

Esto me permitió mejorar mis conocimientos de Java, JavaScript, bases de datos y control de versiones, para poder ir progresando en cada versión y en cada entrega.

En lo personal lo que más se me dificulto fue implementar mis archivos a el trabajo de los demás, ya que fue un código que yo no cree como tal, también dejar listo para el SQL, pero con la ayuda de Luis, fue más fácil.

Gestión / Calendario de Tiempos

Fecha	Inicio	Fin	Interrupción min	Delta T min	Tarea	Comentario
20/10/2026	16:00	21:00	60 - 100	220	Crear calificaciones comenzar con cursos	Tuve un poco de complicación ya que no trabaje por completo cursos.
29/10/2026	18:00	22:00	30- 80	185	Complete la parte de cursos y ayude con promedio y lo aplique a mis archivos.	Para poder integrar calificaciones y notas, también necesitaba arreglar el promedio.
01/11/2026	16:00	22:00	60- 90	285	Pude empezar con la parte de parches y trabajar un poco en usuarios.	La completa implementación de SQL, nos genero 34 errores, pero los logramos arreglar junto a Luis
04/11/2026	16:00	20:00	30-60	195	Pudimos terminar el programa para poder realizar la entrega #5, termine la parte de usuarios.	Ya después de completada la parte de usuarios, tenemos lista esta entrega.

Recuento de Requisitos Funcionales

Andreh Alexander Callejas Ajin

Entrega	Avance Realizado	Resultado acumulado
Fase 3	Diseño del formulario web y validación inicial de los campos.	Pantalla de acceso para usuario y contraseña, integrada al estilo del proyecto.
Fase 4	Conexión del formulario con Java y autenticación mediante MySQL.	El controlador revisa las credenciales y crea una sesión cuando el acceso es correcto.
Fase 5	Integración del registro, cierre de sesión y revisión del flujo.	Se documentaron pruebas de acceso, registro y cierre de sesión.

Planificación

RF01 – Inicio de sesión. Responsable: Andreh Alexander Callejas Ajin

Tarea	Descripción	Prioridad
Acceso y sesiones	Conectar la pantalla con Java, validar las credenciales y permitir cerrar sesión.	Alta
Registro y pruebas	Guardar cuentas en MySQL y revisar datos vacíos, incorrectos y correctos.	Alta

Detalle de la implementación y mi aporte

Mi responsabilidad principal fue el RF01, correspondiente al inicio de sesión. Trabajé en la pantalla que usa el estudiante, en el envío de sus datos al servidor Java y en la conexión de ese flujo con la autenticación y las sesiones. También hice ajustes de interacción en el panel principal y conecté la opción para cerrar sesión. Las funciones de tareas, notas, cursos y promedios pertenecen a los módulos trabajados por mis compañeros.

Pantalla de acceso

La pantalla se construye con HTML, CSS y JavaScript. En **index.html** están el formulario de ingreso y el formulario para crear una cuenta. Para iniciar sesión se solicitan el usuario y la contraseña; para crear una cuenta también se pide el carné. En **estilos.css** definí la presentación, los espacios, los colores y los ajustes para distintos tamaños de pantalla. Agregué transiciones sencillas para que la entrada de la tarjeta y la interacción con los botones se sientan más suaves, sin distraer del formulario.

El archivo **app.js** controla lo que ocurre al enviar los formularios. Primero evita que la página se recargue y comprueba que los campos requeridos tengan información. Si falta un dato, muestra un aviso y no envía la solicitud. Si los campos están completos, envía la información al servidor y muestra el mensaje que recibe. Cuando el acceso es correcto, espera un momento y abre el panel principal. En el registro, si la cuenta se crea correctamente, el formulario vuelve al inicio de sesión y deja escrito el nombre de usuario.

Conexión con Java

La clase **Main** prepara los servicios de MySQL, crea el ControladorLogin y levanta el servidor web en el puerto 8080. **ServidorWeb** sirve los archivos de la interfaz y recibe las solicitudes del navegador. Para mi flujo usa tres rutas: **/api/login** para iniciar sesión, **/api/usuarios** para registrar una cuenta y **/api/logout** para cerrar la sesión. El servidor pasa cada solicitud a la parte que le corresponde y devuelve una respuesta que la pantalla puede mostrar.

Cuando llega una solicitud de acceso, **ControladorLogin** pide al repositorio de usuarios que compruebe las credenciales. En la versión integrada, el repositorio consulta MySQL. Si el usuario no existe o la contraseña no coincide, se rechaza el acceso y no se crea una sesión. Si los datos son correctos, el controlador obtiene el usuario y genera un token aleatorio para identificar esa sesión.

Datos y protección de la contraseña

La clase **Usuario** representa el nombre de usuario, la contraseña y el carné. La clase **Seguridad** convierte la contraseña a un hash SHA-256 antes de guardarla; por eso la base contiene el valor convertido y no la contraseña escrita por la persona. Durante el ingreso se convierte la contraseña recibida de la misma manera y se compara con la almacenada. En esta entrega se utilizó SHA-256 como parte del proyecto académico; un sistema real debería usar un método diseñado específicamente para almacenar contraseñas.

La conexión a MySQL se comparte con los demás módulos del equipo. Mi aporte fue conectar el inicio de sesión con esa estructura para que la cuenta registrada pueda identificarse al entrar. El carné se guarda como dato de la cuenta, pero el acceso se comprueba con el nombre de usuario y la contraseña.

Sesiones y cierre de sesión

Después de validar las credenciales, el servidor guarda el token de sesión en una cookie llamada **student_session**. La cookie se configura para que JavaScript del navegador no pueda leerla directamente. El controlador conserva la sesión mientras está activa y la vence después de 30 minutos sin actividad. Al elegir “Cerrar sesión”, el panel envía una solicitud a **/api/logout**; el servidor elimina la sesión, borra la cookie y devuelve al usuario a la pantalla inicial.

Ajustes del panel principal

Además de la pantalla de acceso, completé controles visuales del dashboard: navegación entre las secciones de tareas, notas y cursos, apertura y cierre del panel de notificaciones y conexión del botón de salida. Mi trabajo en esta parte fue dejar funcional la navegación y el cierre de sesión. La consulta y el registro de información dentro de cada módulo corresponden a las implementaciones de los compañeros responsables de esas funciones.

Archivos principales de mi parte

- **src/main/resources/public/index.html**: formularios de acceso y registro.
- **src/main/resources/public/css/estilos.css**: diseño y animaciones sencillas del login.
- **src/main/resources/public/js/app.js**: revisión de campos, solicitudes y mensajes.
- **src/main/java/Main.java** y **src/main/java/servidor/ServidorWeb.java**: inicio del sistema y rutas usadas por la interfaz.
- **src/main/java/controlador/ControladorLogin.java**: validación de credenciales y manejo de sesiones.
- **src/main/java/modelo/Usuario.java**, **src/main/java/util/Seguridad.java** y **src/main/java/persistencia/RepositorioUsuariosMySQL.java**: datos de la cuenta, conversión de contraseña y consulta de usuarios.
- **src/main/resources/public/dashboard.html**, **css/dashboard.css** y **js/dashboard.js**: controles del panel y cierre de sesión.

Revisión del funcionamiento

Dejé un verificador para el login que compila el proyecto, inicia el servidor y permite comprobar con una cuenta registrada que una contraseña válida sea aceptada y una incorrecta sea rechazada. También preparé una revisión de los controles del dashboard y pruebas del endpoint de cierre de sesión. Los casos documentados incluyen campos vacíos, usuario inexistente, contraseña incorrecta, acceso correcto, registro incompleto o repetido y cierre de sesión. Para las pruebas que usan MySQL se necesita que la base esté disponible y configurada.

En resumen, el recorrido de mi parte es: el estudiante completa el formulario; JavaScript revisa los campos; ServidorWeb recibe la solicitud; ControladorLogin pide validar los datos en MySQL; si coinciden se crea la sesión y se abre el dashboard; al salir, la sesión se elimina y vuelve al login.

Recuento de Requisitos Funcionales

Manuel Rio – 16639

Entrega	Avance Realizado	Resultado acumulado
Fase 3	Asignación del RF12 – Calcular promedio. Definición de la carpeta modulos/promedio dentro de la estructura del proyecto.	Módulo de promedios definido y ubicado en la arquitectura, a la espera de las calificaciones de RF03.
Fase 4	RF12 – Calcular promedio: regla de cálculo, CalculadoraPromedios, ResultadoPromedio, ControladorPromedios, pruebas, promedios.js, tarjeta del dashboard y ruta /api/promedios. Integrado a main (PR #6).	Promedio por curso y promedio general visibles en el dashboard, calculados con las calificaciones de RF03.
Fase 5	Verificación con MySQL y sesiones por navegador. Corrección de la tarjeta para que se actualice al registrar una nota, pruebas ampliadas de 9 a 15 casos y documentación del módulo. Integrado a main (PR #14).	Promedio con datos persistentes en MySQL, por usuario de sesión, que se actualiza sin recargar la página. El ranking de notas reutiliza /api/promedios.

Planificación

RF12 – Calcular promedio. Responsable: Manuel Rio

Tarea	Descripción	Horas est.	Fecha probable	Prioridad
Definir regla de cálculo	Decidir cómo se calcula el promedio por curso y el promedio general, con escala y redondeo.	0.5	14/10/2026	Alta
Calculadora de promedios	Hacer y probar CalculadoraPromedios y ResultadoPromedio usando BigDecimal.	1	14/10/2026	Alta
Controlador de promedios	Hacer ControladorPromedios conectado a RepositorioCalificaciones de RF03.	1	14/10/2026	Alta

Pruebas Java	Crear PruebasPromedios con casos de varias notas, sin notas, redondeo y separación de estudiantes.	1	15/10/2026	Alta
Interfaz del promedio	Crear promedios.js y conectar la tarjeta “Este es tu promedio” del dashboard.	1	15/10/2026	Alta
Ruta del servidor	Agregar GET /api/promedios en ServidorWeb usando el mismo repositorio que calificaciones.	1	15/10/2026	Alta
Prueba en navegador	Registrar notas de prueba y comprobar el resultado en la tarjeta.	0.5	15/10/2026	Media
Actualizar al registrar nota	Hacer que la tarjeta recalcule el promedio cuando se guarda una calificación, sin recargar.	0.5	05/11/2026	Alta
Ampliar pruebas	Agregar casos de punteos distintos, redondeo HALF_UP, notas sin modificar y errores del repositorio.	0.5	05/11/2026	Media
Documentación	Comentarios en las clases, README del módulo y docs/pruebas/promedios.md.	1	05/11/2026	Media

Calendario de planificación

Fecha	Tareas a terminar
14/10/2026	Definir regla de cálculo · Calculadora de promedios · Controlador de promedios
15/10/2026	Pruebas Java · Interfaz del promedio · Ruta del servidor · Prueba en navegador
05/11/2026	Actualizar al registrar nota · Ampliar pruebas · Documentación

Implementación del sistema

Manuel Rio

Mi responsabilidad fue el RF12, cálculo y consulta de promedios. El módulo toma las calificaciones que se registran en RF03 (Fabricio), calcula el promedio de cada curso y el promedio general del estudiante, y lo muestra en la tarjeta “Este es tu promedio” del dashboard. No vuelve a registrar notas ni guarda copias: siempre consulta las calificaciones actuales.

Regla de cálculo

- **Promedio de un curso:** (suma de notas obtenidas ÷ suma de punteos máximos) × 100.
- **Promedio general:** promedio simple de los promedios de cada curso.
- **Formato:** escala de 0 a 100, dos decimales y redondeo

Ejemplo: en Matemática 12/15 y 5/5 dan 17/20 = 85.00; en Química 9/10 = 90.00; el promedio general es (85 + 90) ÷ 2 = 87.50. Se eligió sumar puntos porque la clase Calificacion no tiene un campo de peso: el punteo máximo de cada actividad funciona como su peso, así un parcial de 30 puntos cuenta más que un quiz de 5. Si se promediaran porcentajes, Matemática daría 90 y no 85.

Componentes

CalculadoraPromedios: contiene solo la lógica matemática. Recibe una lista de calificaciones, valida que no sea nula, agrupa por curso y devuelve los resultados. No conoce usuarios, base de datos ni HTML, por lo que se prueba de forma independiente.

ResultadoPromedio: objeto inmutable con el resultado de un curso: identificador, nombre, promedio y cantidad de calificaciones.

ControladorPromedios: valida el identificador del usuario, consulta RepositorioCalificaciones y delega el cálculo a la calculadora. Depende de la interfaz del repositorio y no de una clase concreta.

ServidorWeb: agregué la ruta GET /api/promedios, que responde en JSON el promedio general y la lista de cursos. Hice que calificaciones y promedios usen el mismo repositorio para trabajar con las mismas notas.

promedios.js: consulta /api/promedios y muestra el resultado en la tarjeta. No calcula nada; usa textContent en lugar de innerHTML. Si no hay notas muestra “Aún no tienes calificaciones registradas” en vez de un 0, y si falla la consulta muestra un aviso. Se actualiza al cargar la página, al volver a Tareas y al registrar una calificación.

dashboard.html: solo se agregaron dos identificadores a la tarjeta y la carga de promedios.js; el diseño no cambió.

Mejoras en la Fase 5

Actualización al registrar una nota. Al guardar una calificación, la tarjeta seguía mostrando el valor anterior hasta recargar la página. Lo resolví con un evento: calificaciones.js avisa con calificaciones:actualizadas cuando guarda una nota, y promedios.js lo escucha para volver a consultar el promedio. Es el mismo patrón que ya usaba el equipo con cursos:actualizados, así los módulos se comunican sin depender directamente uno del otro.

Pruebas ampliadas. Pasé de 9 a 15 casos. Antes, si se cambiaba el redondeo, ninguna prueba fallaba; ahora se comprueban los punteos máximos distintos, el redondeo HALF_UP, que las notas originales no se modifican, que una nota nueva se refleja sin guardar copias y los errores de una lista o un repositorio nulos.

Documentación. Agregué comentarios con la regla de cálculo en las clases del módulo, actualicé el README de la carpeta promedio, corregí un comentario desactualizado de la ruta

sobre las sesiones y agregué docs/pruebas/promedios.md, que era el único documento de pruebas que faltaba.

Archivos principales de mi parte

- **src/main/java/modulos/promedio/CalculadoraPromedios.java**: lógica del cálculo.
- **src/main/java/modulos/promedio/ResultadoPromedio.java**: resultado de cada curso.
- **src/main/java/modulos/promedio/ControladorPromedios.java**: consulta y coordinación.
- **src/test/java/PruebasPromedios.java**: pruebas del módulo.
- **src/main/resources/public/js/promedios.js**: consulta y muestra en la tarjeta.
- **src/main/java/servidor/ServidorWeb.java** y **dashboard.html**: ruta /api/promedios y conexión de la tarjeta.
- **src/main/resources/public/js/calificaciones.js**: una línea que avisa cuando se guarda una nota.
- **src/main/java/modulos/promedio/README.txt** y **docs/pruebas/promedios.md**: documentación del módulo y de sus pruebas.

Verificaciones

Las pruebas Java usan el repositorio en memoria y registran las notas con ControladorCalificaciones de Fabricio, por lo que comprueban los dos módulos juntos sin necesidad de MySQL ni del servidor.

También se comprobó que las pruebas detectan errores reales: al cambiar a propósito la fórmula o el redondeo de la calculadora, las pruebas fallan con un mensaje claro. La prueba del redondeo HALF_UP se agregó porque antes ese cambio no se detectaba.

Reflexión

Manuel Rio

La primera dificultad fue definir cómo calcular el promedio. Promediar porcentajes parecía lo más simple. Como el modelo de calificaciones no tiene un campo de peso, decidí sumar puntos obtenidos y punteos máximos, y lo dejé comprobado con una prueba que distingue ambas fórmulas.

Varias dificultades fueron de trabajo con Git y con el entorno:

Aprendí que separar la lógica del cálculo del acceso a datos no es solo teoría: cuando el equipo cambió a MySQL, mi módulo siguió funcionando sin modificar la lógica. También aprendí a compilar y revisar lo que subo antes de cada commit, y a coordinar con mis compañeros antes de tocar archivos compartidos como ServidorWeb.

Gestión / Calendario de Tiempos

Fecha	Inicio	Fin	Interrupción (min)	Delta T (min)	Tarea	Comentario

14/10/2026	21:30	21:55	0	25	Análisis RF03 y regla de cálculo	Revisión del proyecto, CalculadoraPromedios y ResultadoPromedio
14/10/2026	21:55	22:35	0	40	Actualizar rama MANUEL	Merge con main; un archivo se subió vacío y se corrigió
14/10/2026	22:35	22:50	0	15	Controlador de promedios	Conectado a RepositorioCalificaciones
15/10/2026	08:05	08:25	0	20	Pruebas Java	La página no abría: se ejecutaba desde la carpeta equivocada
15/10/2026	08:25	08:45	0	20	promedios.js y dashboard	Tarjeta conectada sin cambiar el diseño
15/10/2026	08:45	09:05	0	20	Ruta /api/promedios	Puerto ocupado; prueba en navegador: 87.50
15/10/2026	23:35	23:55	0	20	Preparación de exposición	Resumen y preguntas frecuentes
16/10/2026	00:00	00:45	0	45	Documentación RF12	promedios.md y documento del módulo
05/11/2026	21:05	21:20	0	15	Actualizar repositorio local	Cambio a main con MySQL y sesiones
05/11/2026	22:05	22:15	0	10	Documentación entrega 5	Revisión de la guía y del formato del equipo
05/11/2026	22:15	22:45	0	30	Mejoras al módulo	Corrección de la tarjeta y pruebas de 9 a 15
05/11/2026	22:45	23:20	0	35	Pull Request #14 y verificación	Merge a main y prueba en el navegador con MySQL
Total			0	**295**	**4.92 horas**	

VIDEOS DE LA QUINTA ENTREGA (ENTREVISTAS)

https://drive.google.com/drive/folders/1Vfp2FiFEuM6RJYBmv5GXtsR8-KA-rny7?usp=drive_link